B0 Start here: foundations for data science

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2

B0 Start here: foundations for data science 3

B0-L1 Python essentials for data work 3

B0-L2 pandas essentials 7

B0-L3 The statistics you need for this track 11

Track B best-practice checklist 15

Appendix: solutions and answer keys 16

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B0 Start here: foundations for data science

Roadmap: Prerequisite for all months. The Python, pandas and statistics this track assumes.

B0-L1 Python essentials for data work

Concept

This track assumes you can read and write everyday Python 3.11 or later. Here is the core it relies on.

Values and collections. Numbers are `int` or `float`, text is `str`, and `True` and `False` are `bool`. `None` means "no value". A `list` holds items in order, `[1.5, 2.0]`. A `dict` maps keys to values, `{"R1": 1.5, "R2": 2.0}`. A `tuple` is a fixed group, such as `(lower, upper)`.

Functions take inputs and return a result. Type hints say what goes in and out, and a docstring says what the function does:

```python
def mean_weight(weights: list[float]) -> float | None:
    """Average weight, or None for an empty list."""
    if not weights:
        return None
    return sum(weights) / len(weights)
```

Loops and comprehensions. `for rid, w in weights.items():` walks through a dict's keys and values together. A comprehension builds a new list in one line: `[w for w in weights if w > 0]`.

f-strings put values into text: `f"{name}: {share:.1%}"` prints `North: 12.5%`. The part after the colon sets the format.

Truthiness. In an `if`, Python treats `0`, `0.0`, `""`, empty collections and `None` as false. That is handy for "is this list empty?", and dangerous for numbers. A share of `0.0` is real data, so test for a missing value with `is None`, never with `if not share`.

Errors. `raise ValueError("negative weight for R2")` stops the function with a message. `try: ... except ValueError:` catches it where you can handle it. Raise early, with a message that names the bad input.

Imports. `import math` or `from pathlib import Path` bring in standard library modules. Install other packages, such as pandas, with pip (lesson B1-L2).

Why it matters

A brand tracker counted brands "with no data" using `if not share:`. Brands with a real 0% share counted as missing, so the report said 14 brands lacked data when only 9 did. The client asked why five brands had vanished from the chart. One `is None` fixed both the count and the chart.

Worked example: weak code

```
1 weights = {"R1": 1.5, "R2": 0.0, "R3": None}
2 total = 0
3 n = 0
4 for k in weights:
2     if weights[k]:
6         total = total + weights[k]
7         n = n + 1
3 print("mean", total / n)
```

1. **Line 1.** Data at module level: nothing else can reuse or test this logic.
2. **Line 5.** Truthiness skips the real 0.0 weight as well as the missing one.
3. **Line 8.** Prints instead of returning, and divides by zero when every weight is missing.

Worked example: strong code

```
1 def mean_weight(weights: dict[str, float | None]) -> float | None:
2     """Mean of the recorded weights, or None when none is recorded."""
2     recorded = [w for w in weights.values() if w is not None]
3     if not recorded:
5         return None
6     return sum(recorded) / len(recorded)
7
8
4 print(f"mean: {mean_weight({'R1': 1.5, 'R2': 0.0, 'R3': None}):.2f}")
```

1. **Line 1.** Type hints show that weights may be missing and that the result may be None.
2. **Line 3.** A comprehension that keeps 0.0 and drops only missing values.
3. **Line 4.** Truthiness is right here: it asks whether the list is empty.
4. **Line 9.** An f-string with a format: two decimals, mean: 0.75.

Put logic in typed functions that return results, use comprehensions to filter, and test for missing values with `is None`.

Recap card

- **Test for a missing number** if value is None. Truthiness also treats 0 and 0.0 as false.
- **Filter a list in one line** A comprehension: [w for w in weights if w > 0].

- **Show 0.125 as 12.5%** f"{share:.1%}".
- **Walk through a dict** for key, value in d.items():
- **Signal bad input** raise ValueError("message naming the input"). Catch it with try/except where you can handle it.

Practice

Exercise 1. Multiple choice (level 1)

`share` is a float, or `None` when no answer was recorded. Which condition is true only when the share is missing?

- a) `if not share:`
- b) `if share is None:`
- c) `if share == 0:`
- d) `if share < 0.01:`

Why?

- r1) Only None is None. A real 0.0 share is a value, and `is None` leaves it alone.
- r2) `is` is faster than `not`.

Exercise 2. Predict the output (level 1)

What does this print?

```
weights = {"R1": 1.5, "R2": 0.0, "R3": 2.0}
kept = [rid for rid, w in weights.items() if w > 0]
print(kept, len(weights))
```

Exercise 3. Fill the gap (level 2)

Complete the function so its type hint allows a missing share, and its label shows a share of 0.125 as `North: 12.5%`.

```
def label(name: str, share: float | _____(1)) -> str:
    if share is None:
        return f"{name}: no data"
    return f"{name}: {share:_____(2)}"
```

Exercise 4. Write the function (level 3)

Write `summarise_weights(weights: dict[str, float | None]) -> dict[str, float | int | None]`. Keys are respondent IDs. Missing weights are `None`. Return a dict with:

- `"n"`: the number of recorded weights, including 0.0,
- `"total"`: their sum (0.0 when none is recorded),

- `"mean"`: total divided by n, or `None` when n is 0.

Raise `ValueError` naming the respondent ID when any weight is negative.

Your code must pass these checks:

- Skip None, count and sum the rest: n 2, total 4.0, mean 2.0.
- A weight of 0.0 is recorded, so it counts in n. Test for missing values with `is None`.
- With no recorded weights, return n 0, total 0.0 and mean None, without dividing by zero.
- Raise ValueError that names the respondent with the negative weight.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Count, total and mean of the recorded weights.
def summarise_weights(
    weights: dict[str, float | None],
) -> dict[str, float | int | None]: ...
```

Starter code:

```python
def summarise_weights(weights):
    total = 0
    n = 0
    for rid in weights:
        if weights[rid]:
            total += weights[rid]
            n += 1
    return {"n": n, "total": total, "mean": total / n}
```

Exercise 5. Spot the bug (level 3)

This function reports 2 brands with no data for `{"A": 0.0, "B": None, "C": 0.4}`. Only one brand has no data. Tap the line with the bug, then fix it.

```python
def count_missing(shares: dict[str, float | None]) -> int:
    """Number of brands with no share recorded."""
    missing = 0
    for brand, share in shares.items():
        if not share:
            missing += 1
    return missing
```

Write down the line number of the bug. Then write the fix.

Sources

- 0, 0.0, an empty string, an empty list and None are falsy; "0" is truthy; f"{0.125:.1%}" gives 12.5%. (verified: Run on Python 3.11 and 3.13.)

B0-L2 pandas essentials

Concept

pandas holds tables as **DataFrames**: named columns over a row **index**. One column is a **Series**. The examples use pandas 3.0, where text columns get the `str` dtype.

```
import pandas as pd

df = pd.read_csv("respondents.csv")
```

Select columns. `df["w"]` returns one column as a Series. `df[["id", "w"]]` returns a smaller DataFrame.

Filter rows. A comparison on a column gives a Series of `True` and `False`, a **mask**. Indexing with it keeps the `True` rows: `df[df["w"] > 0]`. Combine masks with `&` (and), `|` (or) and `~` (not), and put **each condition in brackets**: `df[(df["w"] > 0) & df["in_scope"]]`. Python's `and` raises an error on a Series.

Add columns. `df.assign(w2=df["w"] * 2)` returns a new DataFrame with the extra column and leaves `df` unchanged.

Missing values show as `NaN` or `<NA>`. `df["w"].isna()` marks them. `sum()` and `mean()` skip them.

Group and summarise. `df.groupby("region")["w"].sum()` adds up the weights per region. For several results at once, name each one:

```
df.groupby("region", as_index=False).agg(
    n=("w", "size"),        # rows, missing or not
    recorded=("w", "count"),  # non-missing values
    total=("w", "sum"),
)
```

`as_index=False` keeps `region` as a normal column.

Join tables. `df.merge(regions, on="region_code", how="left")` adds the columns of `regions` to every row of `df`. `how="left"` keeps rows with no match, filling the new columns with missing values. The default, `how="inner"`, drops them.

Tidy the result. `sort_values("region")` orders rows, and `reset_index(drop=True)` renumbers them 0, 1, 2.

Why it matters

An analyst joined respondents to a region lookup with the default inner merge. 312 respondents had a region code missing from the lookup, and

they disappeared from every regional table without an error. The national total no longer matched the sum of the regions. A left merge, followed by a count of rows with a missing region, shows the problem in one line.

Worked example: weak code

```
1  import pandas as pd
2
3  df = pd.read_csv("respondents.csv")
1  df = df.merge(pd.read_csv("regions.csv"), on="region_code")
5  out = {}
2  for i in range(len(df)):
7      r = df.iloc[i]["region"]
3      out[r] = out.get(r, 0) + df.iloc[i]["w"]
```

1. **Line 4.** The default inner merge silently drops respondents with an unknown region code.
2. **Line 6.** A Python loop over rows: slow, and it hides what the code computes.
3. **Line 8.** A missing weight (NaN) turns that region's total into NaN.

Worked example: strong code

```
1  import pandas as pd
2
3
4  def region_totals(respondents: pd.DataFrame, regions: pd.DataFrame) ->
       pd.DataFrame:
5      """Respondents and weight total per region, keeping unknown codes."""
1      joined = respondents.merge(regions, on="region_code", how="left")
7      return (
2          joined.groupby("region", as_index=False, dropna=False)
3          .agg(n=("w", "size"), total_weight=("w", "sum"))
10         .sort_values("region")
11         .reset_index(drop=True)
12     )
```

1. **Line 6.** A left merge keeps every respondent. Unknown codes get a missing region.
2. **Line 8.** dropna=False keeps the missing region as its own group, so nobody disappears.
3. **Line 9.** Named results in one pass. sum skips missing weights.

Filter with bracketed masks, summarise with groupby and named aggregations, and merge with how="left" when every row must survive.

Recap card

- **Filter rows on two conditions** df[(df["w"] > 0) & df["in_scope"]]. Each comparison in brackets, & not and.

- **Count rows against count values** size counts rows, count counts non-missing values. sum and mean skip missing values.
- **Several results per group** df.groupby("region", as_index=False).agg(n=("w", "size"), total=("w", "sum")).
- **Join and keep every row** df.merge(other, on="key", how="left"). The default inner merge drops rows with no match.

Practice

Exercise 1. Multiple choice (level 1)

Which line keeps only rows with a positive weight that are in scope?

- a) `df[(df["w"] > 0) & df["in_scope"]]`
- b) `df[df["w"] > 0 and df["in_scope"]]`
- c) `df[df["w"] > 0, df["in_scope"]]`
- d) `df.filter(df["w"] > 0)`

Why?

- r1) `&` combines two masks row by row, and the brackets make the comparison happen first.
- r2) `&` is shorter to type than `and`.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

What you need to know.

`series.to_dict()` turns a Series into a plain dict, with the index labels as keys. After a groupby, the labels are the group names:

```
pd.Series([3, 5], index=["a", "b"]).to_dict()   # {'a': 3, 'b': 5}
```

```
import pandas as pd

df = pd.DataFrame({"region": ["N", "N", "S"], "w": [1.0, None, 2.0]})
print(df.groupby("region")["w"].sum().to_dict())
```

Exercise 3. Fill the gap (level 2)

Add region names to every respondent, and keep respondents whose region code is missing from the lookup.

```
joined = respondents.merge(regions, on="_____(1)", how="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `region_summary(df: pd.DataFrame) -> pd.DataFrame`. `df` has columns `region` and `w`, where `w` can be missing. Return one row per region, sorted by region, with a default index and the columns:

- `region`,
- `n`: rows in the region, including missing weights,
- `recorded`: rows with a weight,
- `total_weight`: the sum of the weights, 0.0 when none is recorded.

Do not loop over rows.

Your code must pass these checks:

- Return the columns region, n, recorded and total_weight, in that order.
- Count all rows with size, recorded weights with count, and add them with sum. Sort by region and reset the index.
- Leave the input frame unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Rows, recorded weights and weight total per region.
def region_summary(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def region_summary(df):
    out = {}
    for i in range(len(df)):
        r = df.iloc[i]["region"]
        out[r] = out.get(r, 0) + df.iloc[i]["w"]
    return pd.DataFrame({"region": list(out), "total_weight":
    list(out.values())})
```

Exercise 5. Spot the bug (level 3)

For a frame where the only in-scope row has weight 3.0, this function returns two rows, 2.0 and 3.0. Tap the line with the bug, then fix it.

```
import pandas as pd


def in_scope_positive(df: pd.DataFrame) -> pd.DataFrame:
    """Rows that are in scope and have a positive weight."""
    return df[df["w"] > 0 & df["in_scope"]]
```

Write down the line number of the bug. Then write the fix.

Sources

- groupby sum skips NaN; named aggregations with size, count and sum give int64, int64 and float64; a left merge keeps unmatched rows and an inner merge drops them; df[df.w > 0 & df.in_scope] silently ignores in_scope. (verified: Run on pandas 3.0.6 and 3.0.2.)

B0-L3 The statistics you need for this track

Concept

Survey and model results are estimates from a **sample**, used to say something about a **population**. Five ideas carry most of this track.

Mean and spread. The mean is the average. The standard deviation (SD) measures how far values sit from it. The sample SD divides by n - 1. Watch the defaults: NumPy 2's `np.std(x)` divides by n, while pandas 3's `Series.std()` divides by n - 1. For `[2, 4, 4, 4, 5, 5, 7, 9]` they give 2.0 and 2.138.

Standard error (SE). Another sample would give a slightly different estimate. The SE says by how much, on average:

- for a mean: `SD / sqrt(n)`,
- for a share p: `sqrt(p * (1 - p) / n)`.

A share of 50% from 1,000 people has an SE of about 0.0158, 1.6 points. Four times the sample halves the SE.

Confidence interval. A 95% interval is roughly the estimate plus or minus 1.96 SEs: 50% plus or minus 3.1 points. Lesson B8-L2 shows better intervals for small samples and shares near 0 or 100%.

p-value. It is the probability of a difference at least as large as the one you saw, if there were no true difference. It is not the probability that your hypothesis is true. A p-value of 0.03 means "this would be unusual if nothing were going on", nothing more.

Effect size. A tiny difference becomes "significant" in a huge sample. Always report how big the difference is, with its interval, next to any p-value.

Survey weights change all of these. Lesson B8-L1 shows how.

Why it matters

A monthly report flagged every brand whose awareness changed with p < 0.05. With 20,000 respondents a month, a change of 0.6 points was

"significant", and the client reacted to noise-sized moves that no campaign could explain. Reporting each change with its interval, and agreeing in advance on the smallest change worth acting on, ended the false alarms.

Worked example: weak code

```
import numpy as np

scores = np.array([2, 4, 4, 4, 5, 5, 7, 9])
sd = np.std(scores)
print("p < 0.05, so the campaign worked")
```

1. **Line 4.** NumPy's default divides by n, the population SD, not the sample SD.
2. **Line 5.** A p-value says nothing about how big the effect is, or that the hypothesis is true.

Worked example: strong code

```
import math

import numpy as np

scores = np.array([2, 4, 4, 4, 5, 5, 7, 9])
sd = np.std(scores, ddof=1)
se = sd / math.sqrt(len(scores))
low, high = scores.mean() - 1.96 * se, scores.mean() + 1.96 * se
print(f"mean {scores.mean():.2f}, 95% interval {low:.2f} to {high:.2f}")
```

1. **Line 6.** ddof=1 divides by n - 1: the sample standard deviation.
2. **Line 7.** The standard error of the mean.
3. **Line 9.** The estimate with its uncertainty, not a bare yes or no.

Report an estimate with its standard error or interval, know which SD your library computes, and read a p-value for what it is.

Recap card

- **Standard error of a mean and of a share** Mean: SD / sqrt(n). Share: sqrt(p(1 - p) / n). Four times the sample halves the SE.
- **A rough 95% interval** Estimate plus or minus 1.96 standard errors.
- **What a p-value is** The probability of a difference at least this large if there were no true difference. Not the probability that a hypothesis is true.
- **np.std against pandas std** np.std divides by n (population). Series.std divides by n - 1 (sample). Use np.std(x, ddof=1) for the sample SD.

Practice

Exercise 1. Multiple choice (level 1)

An A/B test reports p = 0.03 for the difference in conversion. What does that mean?

- a) There is a 3% chance that the new page is no better.
- b) If the pages truly converted equally, a difference this large or larger would happen about 3% of the time.
- c) The new page converts 3% better.
- d) The result is 97% certain to be correct.

Why?

- r1) A p-value is computed assuming no real difference, and measures how surprising the data would be then.
- r2) A p-value is the probability that the hypothesis is false.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2, pandas 3.0)

```python
import numpy as np
import pandas as pd

x = [2, 4, 4, 4, 5, 5, 7, 9]
print(np.std(x), round(pd.Series(x).std(), 3))
```

Exercise 3. Fill the gap (level 2)

Compute the standard error of a share `p` from `n` respondents, then the lower end of a 95% interval.

```python
se = math.sqrt(p * (1 - p) / _____(1))
low = p - _____(2) * se
```

Exercise 4. Write the function (level 3)

Write `share_ci(successes: int, n: int, z: float = 1.96) -> tuple[float, float] | None`, the normal-approximation interval for a share. With `p = successes / n` and `se = sqrt(p * (1 - p) / n)`, return `(p - z * se, p + z * se)` clipped to [0, 1].

Return `None` when `n` is 0. Raise `ValueError` when `successes` is negative or above `n`.

Your code must pass these checks:

- 500 of 1,000 gives about (0.469, 0.531).

- Clip the interval to [0, 1]: 1 of 10 gives a lower end of 0.0, not a negative number.
- Return None when n is 0, instead of dividing by zero.
- Raise ValueError when successes is below 0 or above n.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Normal-approximation confidence interval for a share.
def share_ci(
    successes: int,
    n: int,
    z: float = 1.96,
) -> tuple[float, float] | None: ...
```

Starter code:

```python
import math


def share_ci(successes, n, z=1.96):
    p = successes / n
    se = math.sqrt(p * (1 - p) / n)
    return (p - z * se, p + z * se)
```

Exercise 5. Architecture design (level 3)

Match each question to the right tool.

Options: SD / sqrt(n), sqrt(p(1 - p) / n), Estimate plus or minus 1.96 SE, The size of the difference, with its interval.

1. How precise is the mean score of 100 respondents with SD 4?
2. How precise is a 50% awareness share from 1,000 respondents?
3. Give a range that is likely to contain the true share
4. With p < 0.001 from 50,000 respondents, does the change matter?

Sources

- np.std([2, 4, 4, 4, 5, 5, 7, 9]) is 2.0 and pd.Series(...).std() is 2.138; np.std with ddof=1 matches pandas. (verified: Run on NumPy 2.4.6 with pandas 3.0.2 and 3.0.6.)
- SE of a 50% share from 1,000 is 0.0158; from 4,000 it is 0.0079. (verified: Computed in Python 3.11.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B0-L1 Python essentials for data work

B0-L1 exercise 1

Hints. (1) Which of these is also true for 0.0? (2) 0.0 is falsy. (3) Only one test checks for None itself.

Answer. b) `if share is None:` Because: r1) Only None is None. A real 0.0 share is a value, and `is None` leaves it alone.

`is None` is true only for a missing value. Truthiness lumps 0.0 in with None.

Common wrong answers.

- `a`: 0.0 is falsy, so `not share` is also true for a real 0% share.
- `c`: That catches a real 0.0 share, and comparing None with < would fail in the next option.
- `d`: `None < 0.01` raises a TypeError, and small real shares would count as missing.
- `b|r2`: Right condition, wrong reason. It is about meaning, not speed.

B0-L1 exercise 2

Hints. (1) `.items()` gives each key with its value. (2) Which weights are above 0? (3) The comprehension does not change the dict.

Answer.

```
1 ['R1', 'R3'] 3
```

The comprehension builds a new list of keys whose weight is positive. The dict itself still has three entries.

Common wrong answers.

- `['R1', 'R2', 'R3'] 3`: The comprehension keeps only IDs whose weight is above 0. R2 has 0.0.
- `['R1', 'R3'] 2`: `len(weights)` counts the original dict, which still has three entries.
- `[1.5, 2.0] 3`: The comprehension collects `rid`, the key, not `w`.

B0-L1 exercise 3

Hints. (1) Which value means missing? (2) A percentage format with one decimal. (3) `None` and `.1%`.

Answer.

```python
def label(name: str, share: float | None) -> str:
    if share is None:
        return f"{name}: no data"
    return f"{name}: {share:.1%}"
```

`float | None` documents that the share may be missing. `.1%` multiplies by 100, rounds to one decimal and adds `%`.

Common wrong answers.

- g1 = `None`; g2 = `.1f`: `.1f` prints 0.1. The `%` format multiplies by 100 and adds the sign.
- g1 = `0`; g2 = `.1%`: The hint names a type. A missing value has the type None.

B0-L1 exercise 4

Hints. (1) Loop with `for rid, w in weights.items():` and skip `w is None`. (2) Raise `ValueError(f"negative weight for {rid}")` before you keep a weight. (3) Compute the mean only when n > 0. Otherwise it is None.

Answer.

```python
def summarise_weights(weights: dict[str, float | None]) -> dict[str, float | int | None]:
    """Count, total and mean of the recorded weights.

    Args:
        weights: Respondent ID to weight, None when missing.

    Returns:
        A dict with n, total and mean. mean is None when no weight is recorded.

    Raises:
        ValueError: If any weight is negative.
    """
    recorded: list[float] = []
    for rid, w in weights.items():
        if w is None:
            continue
        if w < 0:
            raise ValueError(f"negative weight for {rid}: {w}")
        recorded.append(w)
    total = float(sum(recorded))
    n = len(recorded)
```

```
    mean = total / n if n > 0 else None
    return {"n": n, "total": total, "mean": mean}
```

Skipping only `None` keeps real zeros, checking n before dividing avoids a ZeroDivisionError, and an error message that names the ID points straight at the bad row.

B0-L1 exercise 5

Hints. (1) Which brands does the function count? (2) Is 0.0 truthy or falsy? (3) `if share is None:`.

Answer. The bug is on line 5.

Fixed version:

```
def count_missing(shares: dict[str, float | None]) -> int:
    """Number of brands with no share recorded."""
    missing = 0
    for brand, share in shares.items():
        if share is None:
            missing += 1
    return missing
```

`not share` is true for 0.0 as well as None. `share is None` counts only the missing value.

Common wrong answers.

- Line 4: The loop is fine: it visits every brand with its share.
- Line 6: Adding one is right. The condition that decides when is wrong.

B0-L2 pandas essentials

B0-L2 exercise 1

Hints. (1) Which operator works row by row on a Series? (2) `and` wants one True or False. (3) Brackets around each comparison.

Answer. a) `df[(df["w"] > 0) & df["in_scope"]]` Because: r1) `&` combines two masks row by row, and the brackets make the comparison happen first.

Masks combine with `&`, `|` and `~`. Bracket each comparison so it runs before the combination.

Common wrong answers.

- `b`: Python's `and` needs a single True or False. On a Series it raises "The truth value of a Series is ambiguous".
- `c`: A comma inside the brackets makes a tuple, which pandas reads as a column label, not a filter.

- `d`: DataFrame.filter selects columns by name. It does not filter rows by a condition.
- `a|r2`: Right line, wrong reason. `&` works row by row, which `and` cannot do.

B0-L2 exercise 2

Hints. (1) Group the rows by region, then add up w. (2) What does sum do with a missing value? (3) It skips it.

Answer.

```
1 {'N': 1.0, 'S': 2.0}
```

`sum` skips missing values, so region N totals 1.0 and S totals 2.0. `.to_dict()` turns the result into a dict keyed by region.

Common wrong answers.

- `{'N': nan, 'S': 2.0}`: pandas' sum skips missing values, so N adds up only 1.0.
- `{'N': 2, 'S': 1}`: That counts rows. sum adds the values.

B0-L2 exercise 3

Hints. (1) Which column do both tables have? (2) Which merge keeps every row of the left table? (3) `region_code` and `left`.

Answer.

```
1 joined = respondents.merge(regions, on="region_code", how="left")
```

`on` names the shared key. `how="left"` keeps every respondent and fills the region with a missing value when there is no match.

Common wrong answers.

- g1 = `region_code`; g2 = `inner`: An inner merge drops respondents with no match, which is what the task forbids.
- g1 = `region`; g2 = `left`: Join on the column both tables share, the code. The name is what you want to add.

B0-L2 exercise 4

Hints. (1) `df.groupby("region", as_index=False)` keeps region as a column. (2) `.agg(n=("w", "size"), recorded=("w", "count"), total_weight=("w", "sum"))`. (3) Finish with `.sort_values("region").reset_index(drop=True)`.

Answer.

```
1 import pandas as pd
2
3
```

```python
def region_summary(df: pd.DataFrame) -> pd.DataFrame:
    """Rows, recorded weights and weight total per region.

    Args:
        df: Respondents with region and w columns.

    Returns:
        One row per region, sorted, with a default index.
    """
    return (
        df.groupby("region", as_index=False)
        .agg(n=("w", "size"), recorded=("w", "count"), total_weight=("w", "sum"))
        .sort_values("region")
        .reset_index(drop=True)
    )
```

`size` counts rows, `count` counts non-missing values and `sum` skips missing values. Named aggregations produce all three in one pass.

B0-L2 exercise 5

Hints. (1) Which runs first, `>` or `&`? (2) `&` binds tighter than `>`, so Python computes `0 & df["in_scope"]` first. (3) Bracket the comparison: `(df["w"] > 0) & df["in_scope"]`.

Answer. The bug is on line 6.

Fixed version:

```python
import pandas as pd


def in_scope_positive(df: pd.DataFrame) -> pd.DataFrame:
    """Rows that are in scope and have a positive weight."""
    return df[(df["w"] > 0) & df["in_scope"]]
```

`&` binds more tightly than `>`, so the line compares `w` with `0 & in_scope`, which is all zeros. The in-scope condition disappears without an error. Brackets fix it.

Common wrong answers.

- Line 5: The docstring describes the right behaviour. The code does not match it.

B0-L3 The statistics you need for this track

B0-L3 exercise 1

Hints. (1) What does the calculation assume? (2) It assumes there is no real difference. (3) It measures how surprising the data would be then.

Answer. b) If the pages truly converted equally, a difference this large or larger would happen about 3% of the time. Because: r1) A p-value is computed assuming no real difference, and measures how surprising the data would be then.

p = 0.03 says the observed difference would be unusual if there were no real difference. It does not give the probability of any hypothesis or the size of the effect.

Common wrong answers.

- `a`: A p-value is not the probability of a hypothesis. It assumes no difference and asks how unusual the data are.
- `c`: The p-value says nothing about the size of the difference. Report the effect and its interval.
- `d`: There is no such certainty in a p-value.
- `b|r2`: Right statement, wrong reason. The p-value is about the data, given no difference, not about the hypothesis.

B0-L3 exercise 2

Hints. (1) Both compute a standard deviation. Do they divide by the same number? (2) NumPy divides by n. pandas divides by n - 1. (3) The sum of squared deviations is 32, and n is 8.

Answer.

```
1 2.0 2.138
```

The squared deviations sum to 32. NumPy divides by 8 (SD 2.0), pandas by 7 (SD 2.138). Pass `ddof=1` to NumPy for the sample SD.

Common wrong answers.

- `2.0 2.0`: pandas' std divides by n - 1 by default, the sample SD, so it is larger than NumPy's.
- `2.138 2.138`: NumPy's std divides by n by default. It gives 2.0 here.

B0-L3 exercise 3

Hints. (1) p(1 - p) divided by the sample size, then the square root. (2) The multiplier for 95%. (3) `n` and `1.96`.

Answer.

```
se = math.sqrt(p * (1 - p) / n)
low = p - 1.96 * se
```

The SE of a share is `sqrt(p(1 - p) / n)`. A 95% interval stretches 1.96 SEs either side of the estimate.

Common wrong answers.

- g1 = `n`; g2 = `2.58`: 2.58 gives a 99% interval. A 95% interval uses 1.96.
- g1 = `n - 1`; g2 = `1.96`: The SE of a share divides p(1 - p) by n.

B0-L3 exercise 4

Hints. (1) Handle `n == 0` first, then check `0 <= successes <= n`. (2) `se = math.sqrt(p * (1 - p) / n)`. (3) Clip with `max(0.0, low)` and `min(1.0, high)`.

Answer.

```
import math


def share_ci(successes: int, n: int, z: float = 1.96) -> tuple[float, float] | None:
    """Normal-approximation confidence interval for a share.

    Args:
        successes: Number of successes, from 0 to n.
        n: Sample size.
        z: Normal multiplier, 1.96 for 95%.

    Returns:
        (lower, upper) clipped to [0, 1], or None when n is 0.

    Raises:
        ValueError: If successes is outside 0 to n.
    """
    if n == 0:
        return None
    if not 0 <= successes <= n:
        raise ValueError(f"successes must be between 0 and {n}, got {successes}")
    p = successes / n
    se = math.sqrt(p * (1 - p) / n)
    return max(0.0, p - z * se), min(1.0, p + z * se)
```

Guarding n and the count first, then clipping, gives an interval that never crashes or leaves [0, 1]. Lesson B8-L2 replaces this approximation with the Wilson interval, which behaves better near 0 and 1.

B0-L3 exercise 5

Hints. (1) A mean or a share? (2) A range or a precision? (3) Significance is not size.

Answer.

- Standard error of a mean: SD / sqrt(n), here 4 / 10 = 0.4.
- Standard error of a share: sqrt(p(1 - p) / n), here about 0.016.
- A 95% interval: the estimate plus or minus 1.96 SE.
- Whether it matters: the size of the effect, which a tiny p-value in a big sample does not show.

Use the SE formula that matches the estimate, turn it into an interval for a range, and judge importance by the size of the effect.

Common wrong answers.

- matters = `interval`: An interval helps, but the question is whether the size of the change matters. Look at the effect itself.
- se_share = `sd_root_n`: A share is a mean of 0s and 1s, but its SD comes from p itself: use sqrt(p(1 - p) / n).